Data Science in the age of AI (Part 2): If everyone can build a model, what are Data Scientists for?

Why the value of data science lies before and after the model.

JOSE PARREÑO GARCIA
OCT 18, 2025

7 3 7 Share

This week I spoke with an engineer who had built an anomaly detection model for one of the services they manage in their spare time. The goal was simple: generate alerts when unusual patterns appeared in the service metrics. Nothing customer-facing, no direct impact on revenue, and a human still in the loop to decide whether to act on each alert.

The risks were minimal. The system had never produced alerts before, so there was nothing to lose by experimenting. And yet, not long ago, even this type of small aiding tool — something predictive but contained — would have needed a data scientist to design the proof of concept. We would have been called in to scope the problem, select the features, and shape the evaluation.

Instead, with a few hours of spare time, some online reading, and a handful of well-crafted prompts to ChatGPT, the engineer had a working model. It was good enough to deploy for internal use, and they had done it without ever asking for help from the data science team.

We are hearing this more often from engineering leadership. They ask how we could put ten times more models into production, or how to help engineers upskill in machine learning so they can build their own. The intent is positive — more models, more automation, more capability — but it also shifts the role of the data scientist.

In my last post, I looked at how LLMs make it possible for almost anyone to build a model. This example takes that idea further. It is not just possible; *it is happening*. Which means the question for data science is changing. If building is easy, **who is making sure those models are built on the right foundations, integrated into systems that work, and delivering the value they are meant to deliver?**

Senior Data Science Lead is a reader-supported publication. To receive new posts and support my work, consider becoming a free or paid subscriber.

Blog series

- ✅ Data Science in the age of AI (Part 1): The machine wrote my code (and that's not the problem)
- 📌 Data Science in the age of AI (Part 2): If everyone can build a model, what are Data Scientists for? - ***(this post!)***
- Data Science in the age of AI (part 3): Prompting is not building AI systems - *(coming soon!)*
- Data Science in the age of AI (part 4): The science we can't afford to lose - *(coming soon!)*
- Data Science in the age of AI (part 5): The future hybrid - *(coming soon!)*

What this blog will cover

- **Anyone can build a model… but then what?** Why a working model is not the same as a working system.
- ***Part* of real work starts *after* the model is built.** What it takes to make models reliable, efficient, and valuable in production.
- **Data scientists work where things are ambiguous.** How we turn vague business needs into solvable technical problems.
- **The overlooked superpower: context engineering.** Why the next advantage lies in shaping what the model sees and understands.

- **The risk: data scientists becoming GenAI babysitters.** How to avoid being reduced to prompt tweaking and output checking.
- **Closing thoughts.** Why our value lies in systems thinking, scientific rigour, and context — not just code.

Let's start!

Anyone can build a model… but then what?

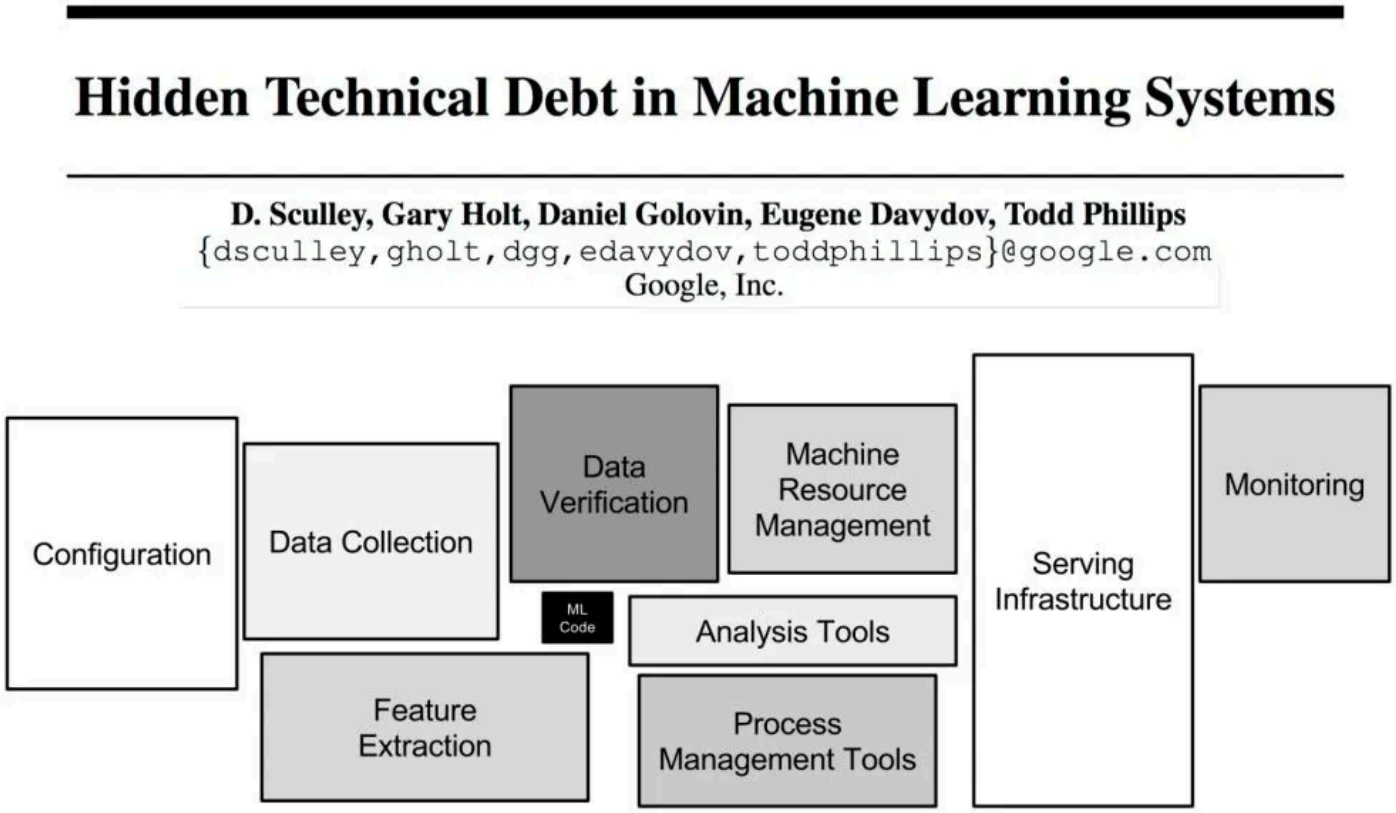
Large language models have made it possible for engineers, analysts, and product managers to build a machine learning model without ever touching a data science team. A few prompts, a handful of code snippets, and a working notebook later — they have something that runs. As we saw in Part 1, even without LLMs, tools like AutoML already allowed people to spin up predictive models with minimal effort. This should not be a surprise. It is a trend that has been building for years.

Now, we should not dismiss this. In many cases, these models will work well enough to pass a quick test. They might even solve the immediate problem, at least on the surface. But there is a gap between something that runs and something that delivers value.

This is the part that often gets overlooked. Shipping a model is not the same as solving the problem. I have spoken with many engineers who see machine learning models as

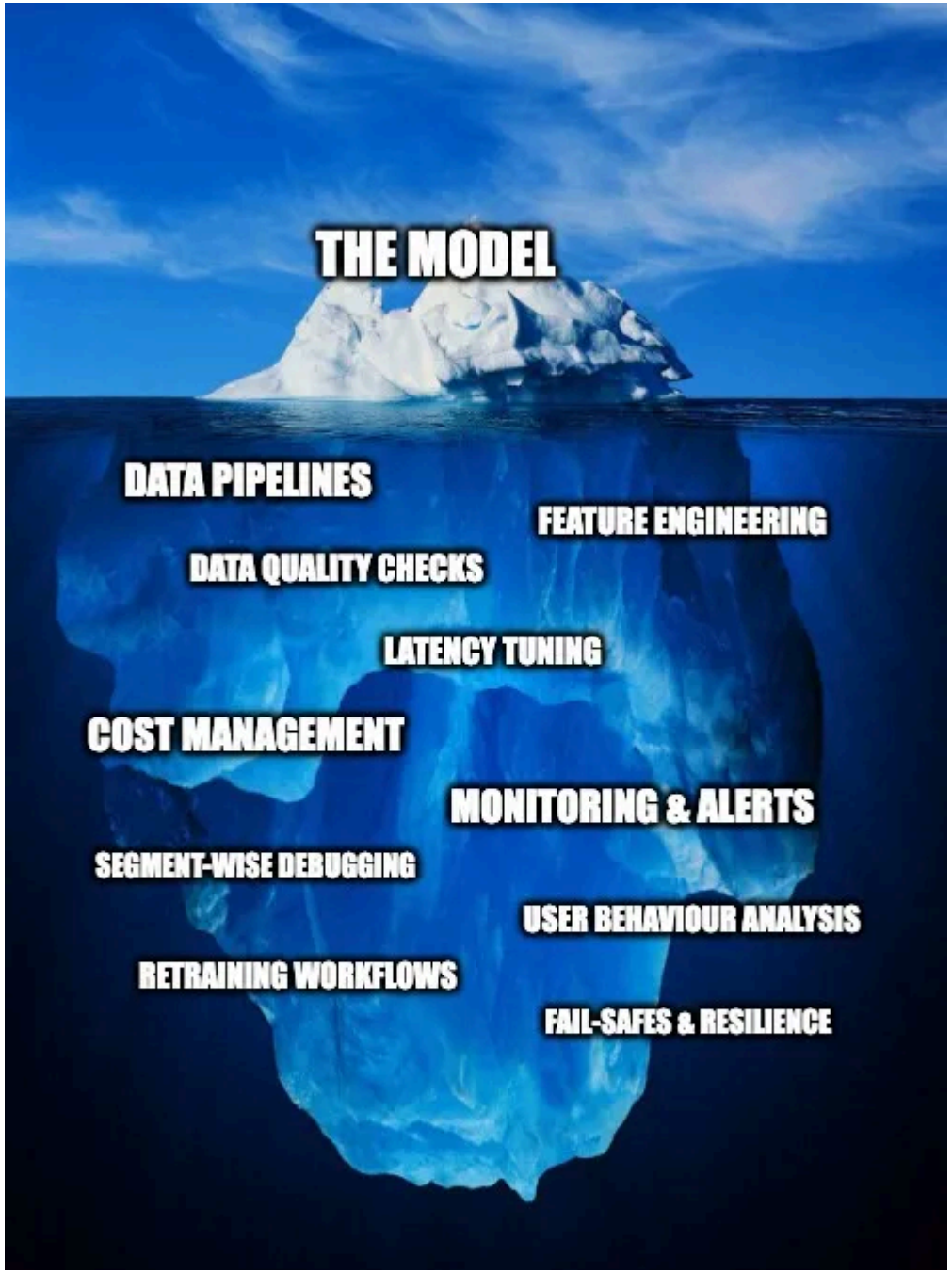
they see any other engineering system. Stripped down to an absurd simplicity, they see ML as just another piece of code.

However, as Data Scientists, we know this is far from true. There is an amazing paper written by Google that exemplifies how complex an ML system is in 1 diagram.

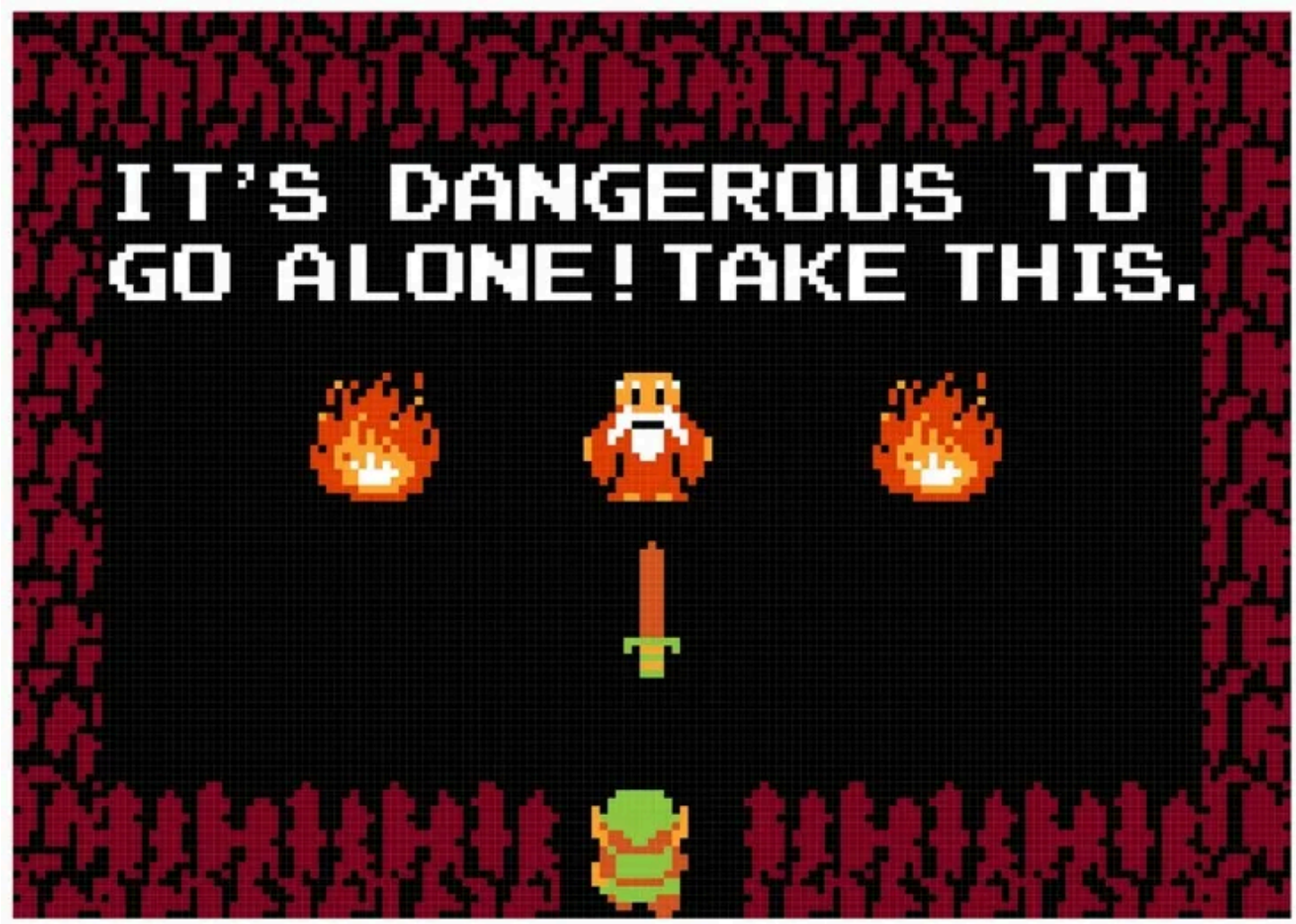


You see the tiny black box in the middle? That is what ChatGPT or AutoML does for you. But what about all the other boxes surrounding it? Ah, now that is a different story. Those boxes are where the real operational work lives.

And that diagram is generous. It assumes the problem is already well defined. In reality, a request like “We need anomaly detection to alert when metric X fluctuates more than normal” is rarely that precise. More often, the problem is messy, underspecified, and entangled with other business constraints.


THE MODEL
DATA PIPELINES
FEATURE ENGINEERING
DATA QUALITY CHECKS
LATENCY TUNING
COST MANAGEMENT
MONITORING & ALERTS
SEGMENT-WISE DEBUGGING
USER BEHAVIOUR ANALYSIS
RETRAINING WORKFLOWS
FAIL-SAFES & RESILIENCE

Which is why the hard question is not *"Can we build it?"*. It is *"Should we build it — and if we do, how do we make sure it works when it matters?"*.



Part of real work starts *after* the model is built.

As data scientists, we know how fun it is to spin up a notebook and treat it as a playground — testing feature engineering ideas, trying multiple models, and plotting

distributions. Similarly, it is also quite fun to see an AutoML tool produce a predictive model faster than your “fun-play-time”. 😅

But, once a model exists, the easy part is over. Turning that model into something that works in production is where much of the effort, risk, and cost actually sit. Production grade ML systems demand far more than accuracy on a test set. They need:

Conditions to run it right

- **Monitoring and alerts** to track data drift, performance degradation, and unusual outputs.
- **Data quality checks** on both the training pipeline, but also, on the data being fed to the model. Believe me, this is time consuming to build, but totally worth the effort.

Knowledge to build it right

- **Segment-wise debugging** to see where the model fails for certain users or markets. Diving into these segments is what makes you stand out as a Data Scientist, because you don’t focus only on high level metrics, you figure out where the model operates well or where it fails.
- **User behaviour analysis** to confirm that model outputs actually influence the product in the intended way.

- **Latency tuning** so results arrive in time to be useful, especially if you are deploying a model for real-time inference. I have seen AutoML produce a model pipeline that took over 1 minute to return a prediction (where our budget was less than 200ms).
- **Feature efficiency** to avoid pulling, transforming, and storing more data than needed. Every new feature you add to the model, is a possible new point of failure. Knowing how to simplify a model without degrading performance is not easy, but it is important.

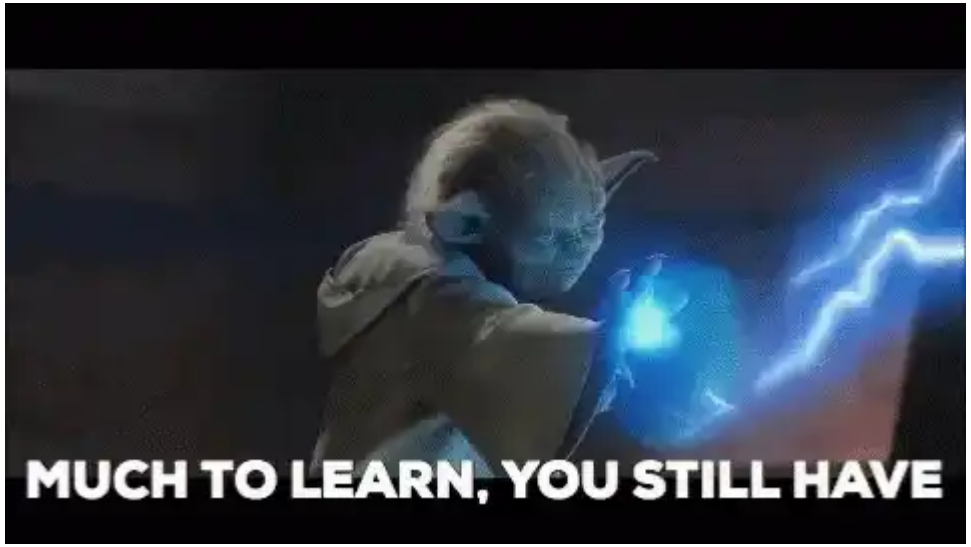
System performance

- **Cost management** so inference does not silently blow up cloud spend. This goes for running the ETL pipelines, the re-training jobs and for inference time.
- **Failsafe options** to handle outages or when your ML model is having too big of an error rate… what to do in those instances?

This is not an exhaustive list, but it does show you some common failure modes. And these happen because the focus stopped at *build* rather than *operate*.

Therefore, if all these things are needed to run machine learning efficiently in production, do we really believe anyone can run ML systems? That ChatGPT can guide you through all the boxes in the diagram? For any high-stakes or critical system,

my view is the same: businesses will still need experts to design, build, and maintain these systems.



Data scientists work where things are ambiguous.

Data science work rarely starts in a neat place. More often, we hear things like:

- “We want to improve conversion… somehow.”
- “Can machine learning help retention?”
- “Could we use AI to summarise customer feedback?”

Our job is to take that ambiguity and turn it into something testable, measurable, and worth building. This means reframing the question so it can actually be solved, scoping the project so it fits the resources and data available, and identifying the trade-offs before any code is written. It also means explaining to stakeholders why *not* building something might be the best decision, or why the solution is simpler than they expect.

We are often brought in *before* anything is scoped. I am not saying that engineering teams never face this, but in my experience, engineers usually receive clearer requirement specs. Their role is to work out how to connect with other teams and systems, and how to build what is asked for, but the requests themselves tend to be relatively deterministic. Data scientists, by contrast, excel at sifting through noise and uncovering the real problem stakeholders want to solve.

This is one of the skills that tools like ChatGPT cannot automate. They can generate code for a problem you describe, but they cannot reliably tell you if you are solving the right problem in the first place.

The overlooked superpower: context engineering.

In traditional machine learning, one of the biggest determinants of model performance is the data you give it. Feature selection, transformations, encoding choices, and segmentation all shape how well the model can learn. Data scientists have

always been the people who understand how to curate those inputs so the algorithm has the best chance to succeed.

Context engineering is the equivalent of feature pre-processing in ML

With large language models, the principle is the same — but the mechanics are different. Instead of engineered features, you are working with *context*. The model's output quality depends not just on the prompt, but on what supporting information it has access to. That includes:

- **Retrieval-augmented generation** (**RAG**) pipelines that pull relevant facts from trusted sources. Some advanced techniques use vector databases (which are based on embeddings, and… surprise surprise… these are things data scientists have been working with for a long time).
- **Structured-to-text translations** that turn tabular or event data into natural language the LLM can understand. LLMs are text machines built around tokens. They are not designed for tabular or numerical data, so we need to bridge that gap if we want them to integrate with these sources.
- **Filtering and selection** to ensure the model does not waste context space on irrelevant or noisy information. One might argue that context space will not be a problem in the future (hey, Gemini is supposed to work with a 1 million token window…). Still, coming from the world of machine learning, we know that noise

can only hurt performance. I can’t see how this wouldn’t be the same if we simply bloat the context window with irrelevant information.

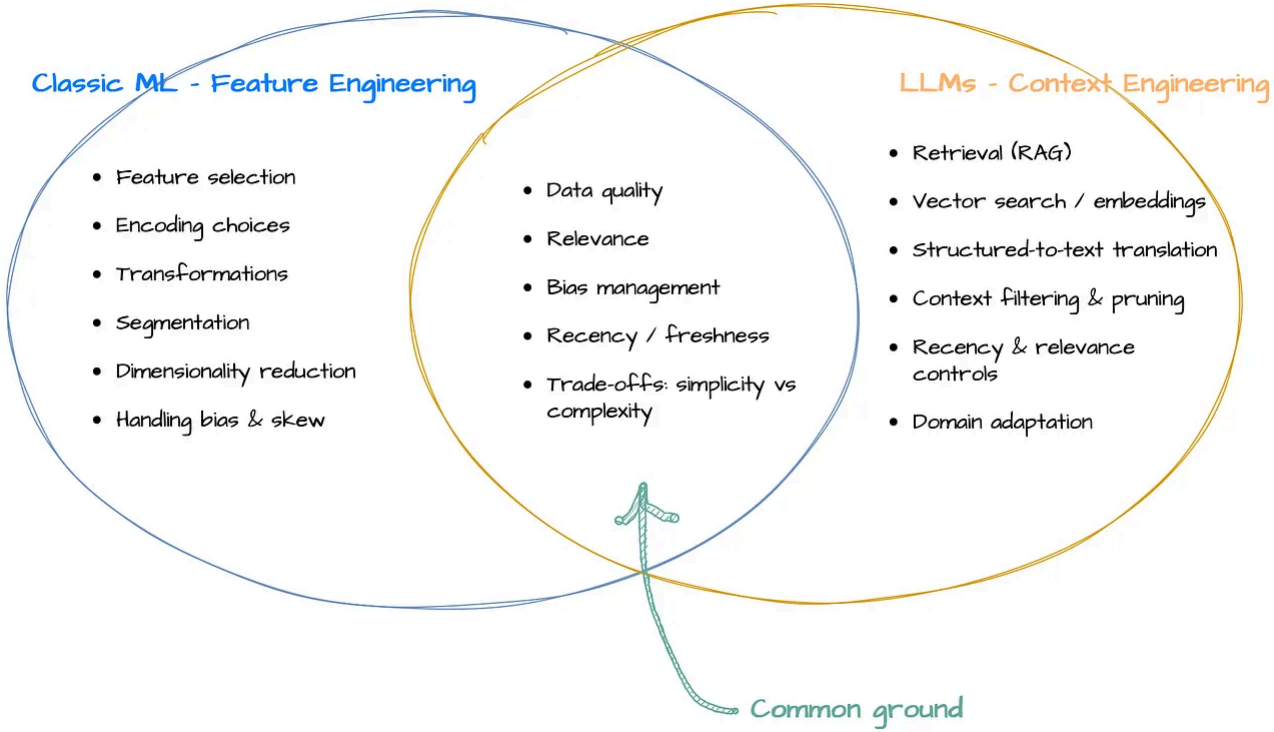
Model-version	Model Provider	Context Window	Output Tokens
o1	Openai	200,000	100,000
o1-mini	Openai	128,000	65,536
o1-preview	Openai	128,000	32,768
gpt-4o	Openai	128,000	16,384
gpt-4o-mini	Openai	128,000	16,384
gpt-4o-realtime-preview	Openai	128,000	4096
gpt-4o-mini-realtime-preview	Openai	128,000	4096
Gemini-1.5 Flash	Google	1 million	8192

Gemini 1.5 Pro	Google	2 million	8192
claude-3.5 sonnet	Anthropic	200,000	8192
claude 3 Haiku	Anthropic	200,000	8192
claude 3 Opus	Anthropic	200,000	8192

Prompt engineering is tactical. Context engineering is strategic.

> Prompt engineering is tactical = anyone can write a clever instruction.
>
> Context engineering is strategic = deciding what the model *knows* when it answers.

That is the part that shapes whether an LLM system is reliable, relevant, and aligned with business needs. And this is where data scientists can excel. We already manage data quality, relevance, recency, and bias for traditional models. We already understand the trade-offs in adding more information versus keeping inputs lean. In a world where LLMs are embedded into products, context engineering is the new feature engineering and it will determine whether those systems perform or fail.



The risk: data scientists becoming GenAI babysitters.

In organisations caught up in GenAI hype, data scientists can quickly find themselves pulled away from their core work. Instead of designing models, running experiments, or integrating systems, we are asked to build the first LLM prototypes, assess whether an agent could replace existing workflows, and handle a stream of ad-hoc requests to *"see if AI can be applied to this problem"*.

I see 3 main risks to consider:

1. **Amnesia. Yes, *"a partial or total loss of memory."*** We know that traditional machine learning already delivers enormous value, but the hype can push companies into a frenzy where that is forgotten. Suddenly, the priority is to work with engineers to build shiny new AI tools. Companies are forgetting why they pay big money to Data Scientists and why they put them into the most critical production systems.
2. **Stakeholders are now skipping the problem-definition stage entirely.** I have been approached with fully-formed "solutions" based on what someone has read in a tech blog or imagined AI might do. They want a chatbot, a summariser, or an agent without first asking whether that is the best tool for the job. One real example: *"Can we build a chatbot that connects to our data so we can ask questions about performance?"* The answer is almost certainly yes. But why not start with a simple dashboard that would give 80% of the value? A chatbot is a different beast, much harder to validate, trust, and verify.
3. **Being pulled into someone else's system *after* it has been built.** Increasingly, we are asked to advise on controlling outputs or improving data quality for LLM tools we had no part in designing. As if "knowing about AI" makes us omniscient about every technical service in the organisation. In reality, it is incredibly hard to fix a system from the outside, especially when you do not know its data sources, constraints, or intended use cases.

The cost of this shift is real

The 3 bullets points above might sound as if I am ranting. I am not. I do believe that Data Scientists need to evolve in this new AI age, but if we forget what our core value add has been, then I do believe that this shift erodes the core of what makes a data science team effective. When the bulk of your time is spent on surface-level fixes, the following happens:

- **Loss of statistical rigour** as time that should go into deep evaluation and controlled experimentation is spent patching outputs and tweaking prompts.
- **Erosion of modelling excellence** as fewer people focus on solving meaningful predictive problems and advancing core ML capabilities.
- **Weak monitoring** because nobody is designing proper metrics or guardrails to track the real-world performance of LLM systems.
- **Demoralisation** as skilled data scientists are reduced to reactive support roles, watching their influence over the product fade.

There is a better alternative.

Instead of being the last link in the chain, data scientists should be involved in designing hybrid systems that combine LLMs with classic ML and business rules. We should be the ones defining how the system is evaluated, monitored, and improved

over time. That means moving upstream — influencing how these tools are scoped, what data they use, and how they are integrated — rather than firefighting their outputs after the fact.

LLMs can and should amplify data science work. But if all we do is clean up after them, we risk losing the very skills that made us valuable in the first place.

Closing thoughts.

If large language models make it possible for almost anyone to build a model, then good. Let them. The more people who can experiment with predictive and generative systems, the more ideas will surface. But building a model is not the end of the journey. It is the “easy” middle step.

Data scientists still play a critical role before the first line of code is written, when the problem is messy and undefined. We also matter after the model exists, when the hard operational work begins. And in this new landscape, we can lead in shaping what models actually *know*: the context that makes them useful, trustworthy, and aligned with the real goal.

I don’t think AI will replace Data Scientists, but if Data Scientists are not careful, the risk is that we will be sidelined into low-value work, cleaning up after tools we did not design. The opportunity is to claim the parts of the system that are hardest to get

right: defining the problem, building it to last, and making sure it performs in the real world.

LLMs can build models. Data scientists build confidence. We design systems that stand up to scrutiny, survive in production, and deliver value long after the first prototype is shipped.

Now, I want to hear from you!

- If anyone can build a model, where do you think data scientists should focus their time to have the biggest impact?
- Have you seen a case where a model was shipped but failed because the hard operational work was skipped? Or a time when clear problem framing completely changed the direction of a project?
- What's your take — is context engineering the next frontier for data scientists, or just another passing trend in the GenAI hype cycle?

Drop your stories, lessons, or counterpoints in the comments. I would love to hear how you are navigating this shift 👇

And see you in the next post about *Data Science in the age of AI.* 👋

Further reading

If you are interested in more content, here is an article capturing all my written blogs!

All my articles, one convenient place

JOSE PARREÑO GARCIA • 15 OCTOBER 2024

Read full story →

7 Likes • 7 Restacks

Discussion about this post

Comments Restacks

Write a comment...

Daniel Popescu / Pluralisk Daniel Popescu 18 Oct

Liked by Jose Parreño Garcia

Wow, the part about engineers building anomaly detection models with LLMs really stood out. I've been using similar quck-prototyping for my cycling data. It's a profound, introspective shift.

LIKED (1) REPLY SHARE

1 reply by Jose Parreño Garcia

Joan Capdevila Pujol 1 Nov

Excellent article! As you say, bringing DS into the problem-definition phase is key. In my experience, involving DS in the evaluation of the AI system from day one is what keeps us engaged throughout.

LIKE REPLY SHARE

1 more comment...

© 2026 Jose Parreño Garcia · Privacy · Terms · Collection notice
Substack is the home for great culture